

MODUL 4

Vom Spec zum ersten Commit.

Genug geredet mit dem Kunden. Heute baust du das Gerüst.

DOZENT

Prof. Dr.-Ing. Jacek Zawisza

PROFESSUR

Betriebliche Informationssysteme & Digitalisierung · FB 3

WAS WIR HEUTE MACHEN.

01

Recap & Brücke

Spec fertig — ab jetzt wird gebaut.

02

Spec-Check

Ist deine Spec überhaupt build-ready?

03

Coding-Agent & Codex

Vom Chat zum Agenten im Terminal.

04

GitHub & gh

Dein Code lebt nicht nur am Laptop.

05

Codex installieren

Einrichten — und einmal live vorführen.

06

Projekt aufsetzen

Repo nach modus-operandi (solo).

07

Hausaufgabe & Ausblick

Backlog, erstes Feature — der Rest zu Hause.

Themen für Bachelor- und *Masterarbeiten.*

Aktuell offene Themen, laufende Arbeiten und eine Auswahl bereits abgeschlossener Arbeiten. Bevor du dich auf ein Thema bewirbst, lies bitte unbedingt die Hinweise zur Anmeldung, Methodik und Zitation.

[HINWEISE LESEN](#)
[SPRECHSTUNDE VIA CALENDLY](#)

Mehr auf prof-z.com

KI & AI-FIRST

-  01 Kernkompetenzen post AI: Was bleibt, wenn KI die Routinearbeit übernimmt?
-  02 KI-Agenten & Substituierbarkeit: Welche Tätigkeiten lassen sich automatisieren?
-  03 AI-First-Unternehmen: Wie stellen sich Firmen intern auf KI um?
-  04 Skill-Datenbanken als Moat: Vertical-KI und verteidigbare Daten-Vorsprünge
-  05 Self-improving Systems: KI, die sich aus Betriebsdaten verbessert
-  06 KI-Adoption in tech-skeptischen KMU: Akzeptanzbarrieren & Change Management
-  07 Rollenwandel post AI: Vom Ausführen zum Orchestrieren
-  08 AI-Native Start-ups: GenAI-Geschäftsmodelle, Wedge & Skalierung

TESLA & AGILE

-  09 Innovationsgeschwindigkeit messen: Tesla vs. VW
-  10 Agile Methoden in der Hardware-Entwicklung: SpaceX & Tesla
-  11 Tesla als Software-Company: Warp-ERP & vertikale Integration

WIRTSCHAFT & WOHLSTAND

-  12 Veränderung der Produktivität durch Innovationen und Impact auf Wohlstand

01

ABSCHNITT

Recap & Brücke.

Die Spec liegt. Jetzt wird daraus eine App.

VON DISCOVERY ZU BUILD

Drei Monate Fragen — heute kippt der Modus.

01

WAS HINTER EUCH LIEGT

Zwei Meetings, 20 Nachrichten, ein KI-Kunde. Ergebnis: eure `spec.md` — Entitäten, Beziehungen, Regeln, Widersprüche.

02

WAS SICH ÄNDERT

Ihr redet nicht mehr mit dem Kunden. Ihr baut. Das Werkzeug wechselt: vom Chat-Fenster zum Coding-Agenten im Terminal.

03

WAS HEUTE PASSIERT

Kein fertiges Produkt. Heute steht das Gerüst: GitHub, Codex, ein echtes Repo — und das erste Stück Code.

KARPATY HATTE RECHT

„Vibe Coding ist passé — jetzt kommt Agentic Engineering.“ (Modul 1, Feb '25)

BISHER — DISCOVERY

Du fragst, der Kunde antwortet.

Chat-Fenster, natürliche Sprache, Anekdoten. Output: ein Dokument.

AB JETZT — BUILD

Du steuerst, der Agent baut.

Echtes Repo, echter Code, echtes Git. Du prüfst, korrigierst, versionierst.

02

ABSCHNITT

Spec-Check.

Garbage in, garbage out — beim Agenten doppelt.

IST DEINE SPEC BUILD-READY?

Bevor du den Agenten loslässt: vier kurze Kontrollfragen.

01

ENTITÄTEN

Jede mit Name, Attributen, Datentypen? Keine Lücke, die der Agent raten muss.

02

BEZIEHUNGEN

1:1, 1:n, n:m sauber benannt?
Mindestens eine n:m ist fast immer dabei.

03

GESCHÄFTSREGELN

Pro Regel ein testbarer Satz?
„Reservierung max. 2h“ —
nicht „muss flexibel sein“.

04

WIDERSPRÜCHE

Aufgelöst, nicht ignoriert?
Was der Kunde
doppeldeutig sagte, steht
jetzt eindeutig drin.

Mantra: Eine vage Spec macht eine vage App. Der Agent füllt jede Lücke — aber selten so, wie du es meinst.

03

ABSCHNITT

Coding-Agent & Codex.

Was das Ding ist — und warum wir Codex zeigen.

WAS IST EIN CODING-AGENT?

Coding-Agent

kein Chatbot, der Code-Schnipsel ausspuckt

Ein KI-Agent, der in deinem Projekt arbeitet: Er liest dein Repo, schreibt Dateien, führt Befehle aus (Tests, Installs) und korrigiert eigene Fehler — mehrere Schritte hintereinander, ohne dass du jeden einzeln tippst.

CHAT vs. AGENT

Chat: Du kopierst Code rein und raus. Der Chat sieht dein Projekt nicht.

Agent: Läuft IN deinem Repo. Sieht alle Dateien, ändert sie direkt, läuft Befehle.

WOMIT IHR ARBEITET

Ein Default, ein Fallback — niemand muss ein Abo kaufen.

01

CODEX — DER DEFAULT

Der Coding-Agent von OpenAI. Läuft über euer **ChatGPT-Abo** — die meisten von euch haben Plus.

Ich führe heute Codex vor.

02

QWEN-POOL — DER FALLBACK

Kein ChatGPT-Abo? Der Kurs stellt ein günstiges Modell (**Qwen3-Coder**) über OpenRouter bereit — genutzt mit Cline/Aider.

03

EIGENES ABO

Cursor, Claude Code, Copilot Pro (gratis via GitHub Education) — erlaubt.
Bedingung: Tool + Modell in der Abgabe dokumentieren.

Codex ohne Plus? Das ChatGPT-Free-Tier reicht zum Starten und Ausprobieren — für eine ganze App eher nicht. Dann: Plus (~20 €/Monat) oder der kostenlose Qwen-Pool.

04

ABSCHNITT

GitHub & gh.

Dein Code lebt nicht nur auf deinem Laptop.

ERST GITHUB, DANN CODE

Versionskontrolle, Backup, Historie — und ein Geschenk für Studis.

GITHUB-KONTO

Kostenlos auf github.com anlegen. Hier lebt dein Repo:
jede Änderung versioniert, nichts geht verloren, du
kommst von überall ran.

GITHUB STUDENT PACK

Mit der HTW-Mail kostenlos freischalten
(education.github.com/pack). Schaltet u.a. Copilot Pro frei
— nimm es mit.

`gh` = GitHub im Terminal. Damit loggst du dich ein und legst dein Repo mit einem Befehl an — gleich auf der nächsten Folie.

Wenn etwas klemmt: Frag die KI mit Fakten.

Die häufigste Frust-Falle: Du klickst „Speichern“, nichts passiert, du klickst nochmal. Stattdessen: die echte Fehlermeldung holen und der KI geben.

1

SPEICHERN

Klick

Du klickst „Speichern“ —
der Spinner dreht endlos.

2

KONSOLE

Öffnen

Browser-Konsole auf:
Cmd+Opt+I oder F12.

3

FEHLER

Kopieren

Die rote Fehlermeldung
wörtlich kopieren.

4

AN DIE KI

Einfügen

Fehler in den Prompt
einfügen: „fix das“.

5

FIX

KI repariert

Die KI findet und behebt
ihren eigenen Bug.

Die KI findet ihre eigenen Bugs — aber nur mit Fakten. „Läuft nicht“ reicht nie, die wörtliche Fehlermeldung schon.

MAC: BREW EINMAL INSTALLIEREN

Nur falls noch kein Homebrew da ist. Windows nutzt winget — diese Folie überspringen.

INSTALLIEREN (EINMALIG)

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
(alternativ auf https://brew.sh/ gehen und den Befehl dort rauskopieren)
```

DANACH — NUR APPLE SILICON (M1-M4)

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile  
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Beim ersten Mal werden die Xcode Command Line Tools mitgezogen (ein paar Minuten). Ohne den Apple-Silicon-Schritt bleibt es bei brew: command not found.

GIT & GH INSTALLIEREN — MAC + WINDOWS

Versionskontrolle + GitHub im Terminal. Danach einloggen.

MAC · HOMEBREW

```
brew install gh git
```

WINDOWS · POWERSHELL

```
winget install --id GitHub.cli
```

```
winget install --id Git.Git
```

DANACH EINLOGGEN (BEIDE)

```
gh auth login
```

```
# → GitHub.com → HTTPS → Login with a web browser
```

```
gh auth status    # zeigt deinen Username = es läuft
```

05

ABSCHNITT

Codex installieren.

Einrichten, einloggen — und einmal live sehen.

NODE & CODEX EINRICHTEN

Erst Node (bringt npm + npx), dann Codex. Mit ChatGPT einloggen.

MAC · NODE (HOMEBREW)

```
brew install node
```

WINDOWS · NODE

```
winget install --id OpenJS.NodeJS
```

DANACH CODEX (BEIDE)

```
npm install -g @openai/codex
```

```
codex
```

```
# → „Sign in with ChatGPT“ wählen → fertig
```

Paketname ist @openai/codex (nicht „codex“ — das ist ein fremdes Altprojekt).

WAS DIE DEMO ZEIGT

Ich lasse Codex ein Mini-Beispiel anfassen — schaut auf den Loop.

1

Repo öffnen

Codex im Projektordner starten — er liest die Dateien selbst ein.

2

Auftrag geben

Eine klare, kleine Aufgabe in natürlicher Sprache — kein Code von mir.

3

Plan ansehen

Codex sagt, was er tun will, BEVOR er es tut. Erst lesen, dann zustimmen.

4

Bauen & prüfen

Er schreibt, führt aus, korrigiert sich. Ich prüfe das Ergebnis — nicht jede Zeile.

06

ABSCHNITT

Projekt aufsetzen.

Ein echtes Repo nach modus-operandi — solo.

DIE STRUKTUR DEINES REPOS

Schlanke Doku, aus der der Agent jederzeit Kontext zieht.

CLAUDE.MD / AGENTS.MD

Kurz-Briefing für den Agenten: Projekt, Stack, Konventionen. Unter 200 Zeilen.

DOCS/SPEC.MD

Deine Spezifikation — bleibt `spec.md`, wird NICHT umbenannt. Die Build-Grundlage.

DOCS/BACKLOG.MD

Features mit IDs, Phase, Status — aus der `spec.md` destilliert (siehe Hausaufgabe).

DOCS/DECISIONS.MD

Kurzes Log: welche Architektur-Entscheidung wann und warum.

Stack-Regel: Framework frei (z.B. Next.js) — **Backend ist SQLite, Pflicht.** Daran hängt die automatische Abnahme (20 Punkte).

REPO ANLEGEN — FÜNF ZEILEN

Ordner, Git, erster Commit, ab auf GitHub.

```
mkdir mein-kundenprojekt && cd mein-  
kundenprojekt  
  
git init  
  
# deine spec.md aus der Discovery hier  
ablegen: docs/spec.md  
  
git add .  
  
git commit -m "chore: initial project setup  
(modus-operandi)"  
  
gh repo create mein-kundenprojekt --private
```

ALTERNATIV

- Legt den Ordner manuell in Windows Explorer / Finder an
- Öffnet den Ordner in VS Code
- Startet Codex und gibt ihm die Anweisung, das Repo anzulegen: z.B. „Erstelle für diesen Ordner ein Projekt namens xy in GitHub“

07

ABSCHNITT

Hausaufgabe & Ausblick.

Den Rest baust du selbst — hier ist der Plan.

Deine Hausaufgabe bis nächste Woche

WAS DU SCHON HAST

Codex eingerichtet, gh eingeloggt, ein leeres Repo mit deiner `spec.md`.

Damit hast du alles, um anzufangen.

WAS ALS NÄCHSTES KOMMT

Inhalte aus der rechten Box umsetzen.

1 · BACKLOG

Codex baut aus `spec.md` ein `backlog.md`: Features mit IDs, Phase, Status.

2 · ARCHITEKTUR

Framework wählen, SQLite-Backend. Codex setzt das Gerüst auf.

3 · ERSTES FEATURE

Ein konkretes Stück, nicht „bau die App“. Plan → Code → testen → committen.

DREI REGELN, DAMIT ES NICHT TEUER WIRD

Ein Agent verbrennt Tokens schneller, als du denkst.

01

KLEINE TASKS

Ein klares Ziel pro Run statt „bau alles“.
Weniger Kontext, weniger Verbrauch,
bessere Ergebnisse.

02

COMMIT NACH GRÜN

Läuft ein Test, committe. Geht der Agent
schief, resettest du — ohne Arbeit zu
verlieren.

03

ERST PLAN, DANN CODE

Lass dir sagen, was der Agent vorhat,
bevor er loslegt. Spart Token und falsche
Wege.

Token-Bewusstsein ist später im Job ein Skill — nicht nur Sparzwang. Wer hier diszipliniert ist, gewinnt.

WAS NOCH KOMMT

Zwei Dinge, die heute bewusst nicht dran waren.

01

QWEN-POOL, WENN NÖTIG

Wer kein ChatGPT-Abo hat oder das Free-Budget aufbraucht:
Setup für den OpenRouter-Qwen-Pool (über Cline/Aider) kommt
per Moodle, sobald er scharf geschaltet ist.

02

KEIN DEPLOYMENT NÖTIG

Deine App läuft lokal — du testest sie selbst. Online-Stellen,
Docker, Cloud-DB sind nicht Teil des Kurses: dein GitHub-Repo
ist die Abgabe.

Nachschlagen: Alle Befehle und der Fallback-Weg stehen in der Build-Anleitung (Moodle: student-build-setup).

PRÜFUNGSLEISTUNG: ZWEI UNITS

Chart
Überarbeiten

Die Vertiefung „Digital Transformation“ (SB52) besteht aus zwei Teilen.

DIGITAL VALUE CREATION · SB52.1

50 Punkte

Der erste Teil der Vertiefung — eigene Veranstaltung, eigene Leistung.

SMART APPLICATIONS · SB52.2

50 Punkte

Spec 15 · Abnahme 20 · Kalibrierung 10 · Prozess 5 — individuell, kein Team, kein Pitch.

Wichtig: Beide Units im **gleichen PZR** ablegen. Die Vertiefung SB52 gilt ab insgesamt mind. 50 Punkten als bestanden.

ALLGEMEINE HINWEISE

Drei Dinge vorab — kurz, aber wichtig.

01

ZWEI VERANSTALTUNGEN

Der Kurs besteht aus Digital Value Creation und Smart Apps — beide gehören zusammen.

02

PC ODER LAPTOP

Wird vorausgesetzt. iPads oder ähnliche Tablets sind nicht geeignet — du brauchst ein echtes Terminal.

03

WINDOWS-TIPP

Nach der Installation einzelner Tools die PowerShell schließen und neu öffnen — sonst wird der neue Befehl nicht gefunden.

Viel Erfolg. Bis nächste Woche.

SMART APPLICATIONS SOMMERSEMESTER 2026

DOZENT

Prof. Dr.-Ing. Jacek Zawisza

PROFESSUR

Betriebliche Informationssysteme & Digitalisierung · FB 3



HTW Corporate Design — Farbpalette.

Die verbindlichen CI-Farben mit Hex-Werten — für manuelle Anpassungen.

GRÜN-FAMILIE

	#4A7500 Dunkelgrün
	#76B900 HTW Grün
	#B5D96A Hellgrün
	#E8F5CC Hellgrün BG

ORANGE-FAMILIE

	#B23F00 Dunkelorange
	#FF5F00 Orange
	#FCB88A Hellorange
	#FFF3E8 Creme-Orange

NEUTRALE

	#1A1A1A Ink
	#333333 Grau
	#666666 Hellgrau
	#999999 Meta-Gray
	#D8D8D8 Rule
	#F5F5F5 Surface
	#FFFFFF Weiß
	#C7C9C3 Meta-Warm

DARK-MODE

	#0E0F0E Dark Deep
	#17181A Dark Base
	#19240F Dark Green Tint
	#1F2124 Dark Raised
	#2A2D31 Dark Elev
	#34373C Dark Rule
	#5A5C57 Dark Sub

Verbindlich: Primärfarbe HTW-Grün #76B900 · Akzente nur aus Grün- oder Orange-Familie · kein Blau.